

מודול 4 — סקריפטינג

חומר לימוד

1 יחידות

1. מודול 4 — סקריפטינג: Bash ו-Python (Scripting for Hackers)

מודול 4 — סקריפטינג: Bash ו-Python (Scripting for Hackers)

מטרות המודול: לרכוש את יכולת האוטומציה הנדרשת בבדיקות חדירה — כתיבת סקריפטים ב-Bash (שפת ה-Terminal) וב-Python (שפת הכלים). נגיע לשני כלים אמיתיים: **Ping Sweep** ב-Bash ו-**Port Scanner** ב-Python.

קבצים: README.md · missions.md · solutions.md · practice.md · slides.md

מבוסס על שיעורי המקור #10 (Bash Scripting) ו-#11–#29 (Python → Port Scanner).

למה סקריפטינג? האקר טוב לא מקליד את אותה פקודה 254 פעמים — הוא כותב לולאה. **Bash** מעולה לאוטומציה מהירה של פקודות מערכת; **Python** היא השפה שבה כתובים רוב כלי האבטחה וה-Exploits. אין צורך להיות מתכנת — מספיק לקרוא, לשנות ולכתוב סקריפטים פשוטים.

תוכן העניינים

חלק א' — Bash 1. מהו סקריפט Bash 2. משתנים וקלט 3. תנאים 4. לולאות 5. פרויקט: Ping Sweep

חלק ב' — Python 6. יסודות וטיפוסים 7. מבני נתונים 8. תנאים ולולאות 9. פונקציות ומודולים 10. פרויקט: Port Scanner

חלק א' — Bash Scripting

4.1 מהו סקריפט Bash?

סקריפט Bash הוא קובץ טקסט עם רצף פקודות Terminal, שרץ בזו אחר זו. כך הופכים סדרת פקודות ידניות לכלי אוטומטי.

```
#!/bin/bash
# bash אומרת למערכת להריץ עם "Shebang" – השורה הראשונה
echo "Hello, Hacker!"
```

הרצה:

```
chmod +x script.sh # הפיכה לניתן-הרצה (זוכרים ממודול 2?)
./script.sh         # הרצה
```

💡 ה-#!/bin/bash הוא **Shebang** — חובה בתחילת כל סקריפט. שורות שמתחילות ב-# (מלבד ה-Shebang) הן הערות.

4.2 משתנים וקלט משתמש

```
#!/bin/bash
name="Kali"          # ==הצבה – ללא רווחים סביב ה-#
echo "Target is $name" # $ שימוש – עם
read -p "Enter an IP: " ip # קלט מהמשתמש
echo "You entered $ip"
```

- **ללא רווחים** סביב ה- = (זו טעות נפוצה: name = "x" שגוי).
- \$name — קורא את הערך. — \${name} — צורה בטוחה יותר בתוך מחרוזות.
- **ארגומנטים משורת הפקודה**: \$1, \$2 = הארגומנט הראשון/שני; \$# = מספרם.

```
#!/bin/bash
echo "First argument: $1" # ./script.sh 10.0.0.5 → 10.0.0.5
```

4.3 תנאים ב-Bash

```
#!/bin/bash
read -p "Enter port: " port
if [ "$port" -eq 80 ]; then
    echo "HTTP"
elif [ "$port" -eq 443 ]; then
    echo "HTTPS"
else
    echo "Unknown"
fi
```

אופרטורים נפוצים (בתוך []): | מספרי | מחרוזות | קבצים | | — | — | — | |
| -f | קובץ קיים || -ne | שונה | != | שונה | -d | תיקייה קיימת || -gt | גדול | -z | ריק | -x | ניתן-הרצה ||
| -lt | קטן ||

4.4 לולאות ב-Bash

```
# על רשימה for לולאת
for host in 10.0.0.1 10.0.0.2 10.0.0.3; do
    echo "Pinging $host"
done

# לולאה על טווח
for i in $(seq 1 10); do
    echo "Number $i"
done

# לולאת while
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    count=$((count + 1))    # Bash-חשבון ב
done
```

4.5 פרויקט Ping Sweep — Bash

כלי אמיתי: סורק תת-רשת ומזהה אילו מכונות **חיות** (Host Discovery) — בדיוק מה שעושים בתחילת שלב הסריקה.

```
#!/bin/bash
# ping_sweep.sh – 24/ רשת מכוונת חיות בתת-רשת
# שימוש: ./ping_sweep.sh 192.168.1

if [ -z "$1" ]; then
    echo "Usage: $0 <network> e.g. $0 192.168.1"
    exit 1
fi

network=$1
echo "[*] Scanning $network.0/24 ..."

for ip in $(seq 1 254); do
    # -c 1 = חבילה אחת, -W 1 = שנייה timeout
    ping -c 1 -W 1 "$network.$ip" > /dev/null 2>&1 && \
        echo "[+] $network.$ip is UP" &
done
wait
echo "[*] Sweep complete."
```

איך זה עובד: 1. בודק שהתקבל ארגומנט רשת (\$1), אחרת מדפיס שימוש ויוצא. 2. לולאה על 1–254 (כל המארחים ב- /24). 3. ping -c 1 לכל כתובת; אם הצליח (&&) — מדפיס שהמכונה חיה. 4. ה- & מריץ במקביל (מהיר יותר); wait ממתין לסיום כולם.

⚠ הרץ רק על תת-הרשת של המעבדה שלך.

Python 4.6 — יסודות וטיפוסים

```
target = "192.168.1.10" # String (מחרוזת)
port = 80                # Integer (מספר שלם)
timeout = 1.5            # Float (עשרוני)
is_open = True           # Boolean
```

• **String** — טקסט במרכאות · **Integer** — מספר שלם · **Float** — עשרוני · **Boolean** — `False / True`.

מחרוזות — פעולות שימושיות לניתוח פלט

```
banner = "Apache/2.4.49"
print(banner.upper())      # APACHE/2.4.49
print(banner.split("/"))  # ['Apache', '2.4.49']
print("2.4.49" in banner) # True
```

f-strings — שילוב משתנים בטקסט:

```
ip, port = "10.0.0.5", 22
print(f"Scanning {ip} on port {port}") # Scanning 10.0.0.5 on port 22
```

4.7 מבני נתונים

```
ports = [21, 22, 80, 443] # List – ניתן לשינוי – מסודר,
ports.append(8080)         # הוספה
print(ports[0], len(ports)) # 21 5

services = {80: "HTTP", 443: "HTTPS"} # Dictionary – מפתח:ערך
print(services[80])             # HTTP

address = ("10.0.0.1", 80) # Tuple – קבוע
```

מבנה	תחביר	ניתן לשינוי?	שימוש
List	[]	כן	רשימת פורטים/יעדים
Dictionary	{k: v}	כן	מיפוי פורט ← שירות
Tuple	()	לא	זוג IP+port קבוע

4.8 תנאים ולולאות

```
port = 22
if port == 22:
    print("SSH")
elif port == 80:
    print("HTTP")
else:
    print("Unknown")

# לולאה על טווח
for port in range(1, 101):      # עד 100
    print(port)

# לולאה על רשימה
for p in [21, 22, 80]:
    print(f"Checking {p}")
```

אופרטורים: `==` `!=` `<` `>` `and` `or` `not`.

4.9 פונקציות ומודולים

```
def scan_port(ip, port):      # הגדרת פונקציה
    print(f"Scanning {ip}:{port}")
    return True

scan_port("10.0.0.1", 80)    # קריאה

import socket # תקשורת רשת – לב הסורק
import sys    # ארגומנטים ומערכת
```

• `socket` — יצירת חיבורי רשת. `sys` — אינטראקציה עם המערכת. התקנת ספריות: `pip install` `<שם>`.

Python — Port Scanner פרויקט 4.10

הכלי המרכזי של המודול — מיישם את **TCP** ואת **לחיצת היד** ממודול 3:

```
#!/usr/bin/env python3
import socket
import sys

target = input("[*] Enter target IP: ") # קלט מהמשתמש
print(f"[*] Scanning {target}")

try:
    for port in range(1, 1025):          # פורטים 1–1024
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # TCP over IPv4
        s.settimeout(0.5)                # אל תחכה יותר מחצי שנייה
        result = s.connect_ex((target, port)) # מחזיר 0 אם פתוח
        if result == 0:
            print(f"[+] Port {port} is OPEN")
        s.close()
except KeyboardInterrupt:
    print("\n[!] Stopped by user")
    sys.exit()
except socket.gaierror:
    print("[!] Hostname could not be resolved")
    sys.exit()
```

שורה אחר שורה: 1. `socket.socket(AF_INET, SOCK_STREAM)` — יוצר חיבור TCP מעל 2. IPv4. `settimeout(0.5)` — מונע היתקעות על פורט חסום. 3. `connect_ex((target, port))` — מנסה את לחיצת היד; מחזיר 0 אם הפורט **פתוח**. 4. הלולאה עוברת על 1–1024; `try/except` מטפל ב-Ctrl+C ובשגיאות DNS.

⚠ **הרץ אך ורק על 127.0.0.1 או מכונות המעבדה שלך.**

Bash מול Python — מתי מה?

Python	Bash	
לוגיקה מורכבת, רשת, פרסינג	אוטומציה של פקודות מערכת	חוזק
טוב לכלים גדולים	מהיר לדברים קצרים	מהירות כתיבה
Port Scanner, Exploits	Ping Sweep, שרשור כלים	דוגמה
Metasploit modules, כלי אבטחה	one-liners, סקריפטי שרת	נפוץ ב

💡 בפועל משלבים: Bash “מדביק” כלים קיימים במהירות, Python בונה כלים חדשים.

סיכום המודול

- **Bash:** Shebang, משתנים, `read`, תנאים (`[ ]`), לולאות (`while / for`) — ובנינו **Ping Sweep**.
- **Python:** טיפוסים, מבני נתונים (`List/Dict/Tuple`), תנאים, לולאות, פונקציות, מודולים (`socket`) — ובנינו **Port Scanner**.
- **Bash** לאוטומציית מערכת מהירה; **Python** לכלים ולוגיקה.

מה הלאה?

→ תרגילים — `missions.md` · פתרונות — `solutions.md` · תרגול ותרחישים — `practice.md` →

מודול 5 — איסוף מידע

← חזרה למפת הקורס